

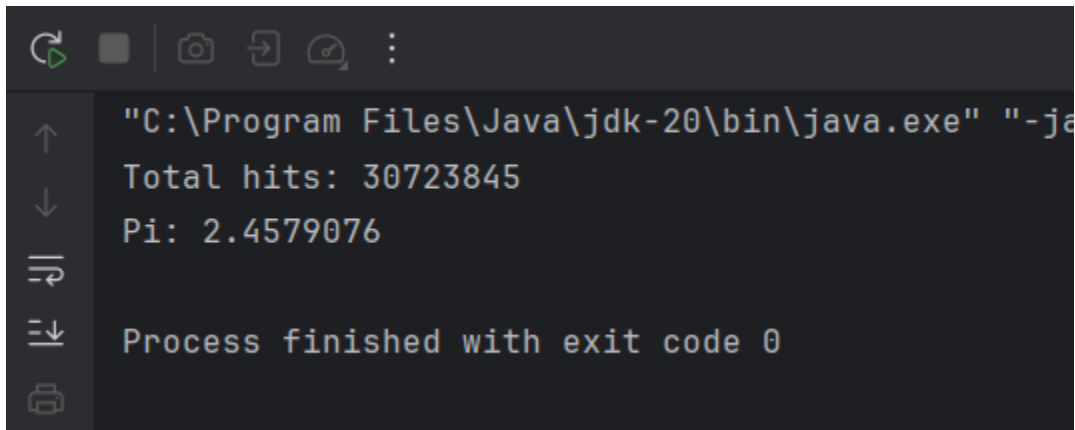
All source code are provided in repository

Task1

The results :

- 1) Total hits: 29927823
Pi: 2.39422584
- 2) Total hits: 33275011
Pi: 2.66200088
- 3) Total hits: 30326301
Pi: 2.42610408
- 4) Total hits: 30723845
Pi: 2.4579076

The output example :



```
"C:\Program Files\Java\jdk-20\bin\java.exe" "-ja
Total hits: 30723845
Pi: 2.4579076
Process finished with exit code 0
```

Task 2

The output :

```
Run Task_2 x
"C:\Program Files\Java\jdk-20\bin\java.exe" "-javaagent:D:\IntelliJ I
Single-threaded:
Pi = 3.14183912
Time = 2.3393095 seconds

4 Threads + AtomicLong:
Pi = 3.14152528
Time = 1.4327519 seconds

Process finished with exit code 0
```

Task 3 :

The code result :

```
--- Benchmark ---
Threads: 1 | Pi: 3,141718 | Time: 4225,23 ms
Threads: 2 | Pi: 3,141346 | Time: 2747,28 ms
Threads: 4 | Pi: 3,141542 | Time: 1056,61 ms
Threads: 8 | Pi: 3,141635 | Time: 647,65 ms
Threads: 16 | Pi: 3,141994 | Time: 558,46 ms
Threads: 32 | Pi: 3,141440 | Time: 496,61 ms

Threads | Runtime (ms) | Speedup | Efficiency
1 | 4225,23 | 1,00x | 100,00%
2 | 2747,28 | 1,54x | 76,90%
4 | 1056,61 | 4,00x | 99,97%
8 | 647,65 | 6,52x | 81,55%
16 | 558,46 | 7,57x | 47,29%
32 | 496,61 | 8,51x | 26,59%

Process finished with exit code 0
```

Threads	Runtime	Speedup	Efficiency
1	4225,23	1,00x	100,00%
2	2747,28	1,54x	76,90%
4	1056,61	4,00x	99,97%
8	647,65	6,52x	81,55%
16	558,46	7,57x	47,29%
32	496,61	8,51x	26,59%

Questions

1. An 8-core CPU can run about 8 threads simultaneously. With 16 threads, multiple threads must share the same cores, causing context-switching and scheduling overhead. Therefore, 16 threads cannot achieve 2× the performance of 8 threads.

2. The synchronized version was slower because all threads competed for the same shared counter. Synchronization/atomic operations introduce contention and waiting, which can outweigh the benefit of using multiple threads.